Git - Version Control System

Day 1

Ionuț Ciocîrlan
Kluger Training

About Me

- Software developer, software architect
- Git user for ~12 years
- Linux admin for ~28 years, using it as daily driver for almost 20
- Teacher / trainer for ~8 years

About the Course

- 2 days, 8 hours a day (9:00 - 17:00)
  - Lunch break (12:30 - 13:30) & short breaks
- Theoretical presentation & practical exercises
- Questions are welcome anytime!
- Materials: https://github.com/xlotlu/git-course

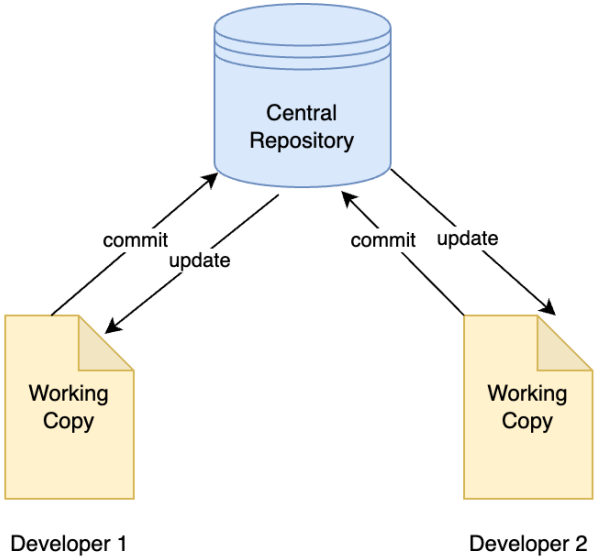
What is Version Control?

- Manage changes to files
- Track their history over time
- Revert to previous states
- Facilitate collaboration among multiple contributors
- Enable experimentation or feature development on isolated branches
- Combines changes into a main branch

Benefits of Version Control Systems

- Enhanced Collaboration
- Historical Insight
- Data Protection and Recovery
- Concurrent Development
- Error Mitigation

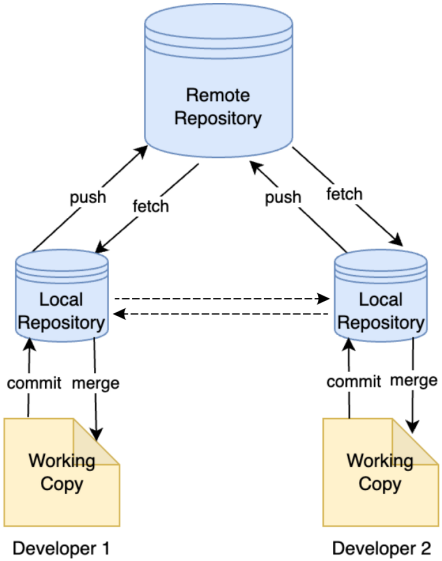
Centralized Version Control Systems



Centralized VCSs (e.g., Subversion)

Advantages	Disadvantages
Straightforward to understand and configure	Single point of failure
Single source of truth for the project	Dependency on connectivity

Distributed Version Control Systems



Distributed VCSs (e.g., Git)

Advantages	Disadvantages
Eliminates single point of failure (all users have a full repository copy)	Steeper learning curve
Offline functionality	Storage overhead
Simplifies branching and merging workflows	

Git History

- **2005**: Created by Linus Torvalds for the Linux kernel, which needed a distributed version control system that was fast, reliable, and open source.
- **2008**: GitHub launched as a platform to host Git repositories, significantly boosting Git's popularity.
- **2010s**: Git became the de facto standard for version control in software development, adopted by major companies and open-source projects worldwide.

Installing Git

- See documentation for system-specific installation details.

Command: `git config`

Set up name and email to author commits:

```
git config set --global user.name "Your Name"
git config set --global user.email "email@example.com"
```

Review configuration entry:

```
git config get user.name
```

Review full configuration:

```
git config list
```

Git Configuration Levels

- **Local:**
  - Specific to a single repository
  - `--local` flag or no flag
- **Global:**
  - Applies to all repositories for the current user
  - `--global` flag
- **System:**
  - Applies to all users on the system
  - `--system` flag

Practice: Configuration

1. Inspect default git configuration before defining any custom setting.
2. Configure your name and e-mail globally.
3. Inspect git configuration again. What's changed?
4. Inspect git configuration at each level.
5. Try getting the value of a single configuration entry.

Initializing a New Repository

- **Command:** `git init`
- Creates a hidden `.git` folder (metadata and history) in your project directory.
- A directory containing the `.git` folder is a fully functional Git repository.

Practice: Initializing a Repository

1. Create a directory named `new_project`.
2. Change current working directory to `new_project`.
3. Run `git init`.
4. Notice the creation of `.git` directory.
5. Check out local git configuration.

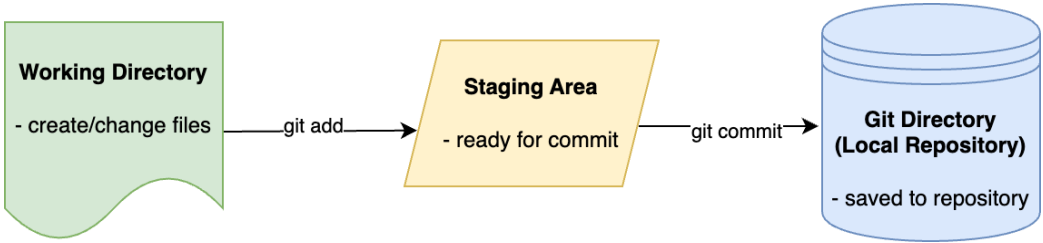
Cloning an Existing Repository

- **Command:** `git clone <repository-url>`
  - Creates a new directory with all the repository content at the current location.
- **Command with custom destination:**
  `git clone <repository-url> <destination-directory>`
  - Clones the repository into a specified target directory.
- Supports **HTTPS** vs **SSH** authentication methods.

Practice: Cloning

1. Clone `git@github.com:xlotlu/git-playground.git` locally into a custom directory (e.g. `my copy of git playground`) outside `new_project` created previously.
2. Navigate inside the custom directory and take a look at the contents.
3. Set `user.name` configuration entry to be `"Mickey Mouse"` at local config level.
4. Inspect local configuration.
5. Navigate outside this directory, and clone the repo above again, this time without specifying a custom name.
6. Navigate inside `git-playground` and check the local configuration. Is Mickey Mouse there?

Sections of a Git Project



Tracked vs Untracked Files

- **Tracked files:** Files that Git knows about and tracks.
- **Untracked files:** Files that Git doesn't track (yet).
- **Command:** `git status` — Provides an overview of your working directory and staging area status.

Practice: File Tracking & Status

1. Go to `new_project` and run `git status`.
2. Create a new file `git_installation.txt`, and edit it with the following content: Practice File Link
3. Run `git status` again.

Tracking Files

- **Command:** `git add <file/dir(s)>`
- Accepts file/directory names or paths.
- If the argument is a directory, all files inside it are added recursively.
- Files are moved to the staging area.
- Changes become ready for commit.

Saving Changes

- **Command:** `git commit -m "Commit message"`
- All staged changes are saved to the Git repository.
- A **commit** consists of:
  - A snapshot of the changes
  - Metadata (author, date, message)
- **Commit message:** Should be concise, imperative, and state a clear rationale.

Inspecting Changes

- **Command:** `git diff`
  - Shows changes that are modified but not yet staged.
- **Command:** `git diff --staged`
  - Compares your staged changes against your last commit.

Practice: Add, Diff, and Commit

1. Run `git diff` to see all unstaged changes.
2. Add `git_installation.txt` to the staging area.
3. Run `git status`, `git diff`, and `git diff --staged`.
4. Save your work with `git commit`. Use a meaningful commit message.
5. It seems like you forgot a step! Edit `git_installation.txt` to include:
   `2. Run the downloaded installer file.`
6. Run `git status`, `git diff`, and `git diff --staged` again. Save your change with a new commit.

Removing Files

- **Command:** `git rm <file>`
  - Used for tracked files.
  - Removes the file from the working directory and stages the deletion for commit.
- **Command:** `git rm --cached <file>`
  - Removes the file from version control (untracks it) while keeping the file locally on your disk.

Practice: Removing Files

1. Create a new file `my_file.txt` and write a few characters in it.
2. Commit this file to the repository.
3. Remove `my_file.txt` with `git rm`. Commit this change.
4. Redo steps 1–2. Remove `my_file.txt` with `git rm --cached`. What's changed? Check the status of the repo. Commit the change.

Moving / Renaming Files

- Git doesn't explicitly track file movement as a distinct operation.
- Simply moving a file (`mv <old> <new>`) won't be immediately registered as a rename in Git.
- **Command:** `git mv <old> <new>`
  - Correctly registers the file as renamed or moved within Git.

Practice: Renaming Files

1. Rename `git_installation.txt` to `git_installation_windows.txt` (outside of Git). Check the status of your git repo.
2. Rename `git_installation_windows.txt` back to `git_installation.txt`. Check the status of your git repo once again.
3. Rename `git_installation.txt` to `git_installation_windows.txt` using the appropriate git command. Commit the change.

Viewing Commit History

- **Command:** `git log`
  - Displays a list of commits in reverse chronological order.
  - Each commit displays: hash, author, date, and commit message.
  - **Commit hash:** Unique identifier for a commit.
- **Format Options:** `--oneline`, `--graph`
- **Filtering Options:** `-n`, `--since`, `--until`, `--author`

Practice: Commit History

1. Run `git log` in your existing repositories.
2. Clone a new repo with a richer history (e.g. https://github.com/psf/requests) and try out different formatting / filtering options for `git log`.

Viewing Changes Over Time

- **Command:** `git show <commit-hash>`
  - Displays the changes introduced in a specific commit.
- **Command:** `git diff <older-commit-hash> <newer-commit-hash>`
  - Shows all changes introduced between `<older-commit-hash>` and `<newer-commit-hash>`.

Practice: Viewing Commit Diffs

1. Inspect changes introduced by a specific commit in one of your repositories.
2. View changes between two commits in one of your repositories.

Undoing Things

- Amending commits
- Unstaging staged files
- Unmodifying modified files

Amending Commits

- **Command:** `git commit --amend`
  - Modifies the commit message of the latest commit.
  - Adds files that were previously forgotten to the last commit.
  - Alters commit history.

Unstaging Files

- **Command:** `git restore --staged <file>`
  - Removes the specified file from the staging area.
  - Does not affect local changes made in the file.

Unmodifying Files

- **Command:** `git restore <file>`
  - Discards all unstaged changes in the file.
  - Returns the file to its last committed state.
  - **Irreversible action!**

Practice: Undoing Changes

1. Add the following file: git_installation_linux.txt to the repository and commit the change with the message: `"Add installation steps for Linux"`.
2. You decide you want to include the installation steps for macOS in the same commit. Add git_installation_mac.txt to the repo (and staging area). Amend the last commit to include this file and change the message to `"Add installation steps for other operating systems"`.
3. Add some extra lines at the end of `git_installation_mac.txt`. Stage the file. View status.
4. You decide you don't want to commit these changes yet, so you remove them from the staging area. View status again.
5. You decide you want to completely drop these changes, so you restore `git_installation_mac.txt` to its last committed state.

Branching

- **Branch:** A lightweight pointer to a specific commit.
- Allows developers to work on isolated changes without affecting the main project branch.
- **Benefits:**
  - Parallel development
  - Safe experimentation
  - Simplified collaboration

Working with Branches

- **Create a new branch:** `git branch <branch-name>`
- **View all branches:** `git branch`
- **Switch to another branch:** `git switch <branch-name>`
- **Create and switch in one command:** `git switch -c <branch-name>`

Practice: Branching

1. Create a branch `linux-installation` and switch to it. Edit `git_installation_linux.txt` to replace steps 2–4 with the content from steps_linux_alternative_1.txt. Commit the change.
2. Switch branch back to `main`. Create and switch to `refactor` branch. Edit `git_installation_linux.txt` to replace steps 2–4 with the content from steps_linux_alternative_2.txt. Commit the change.

Merging Branches

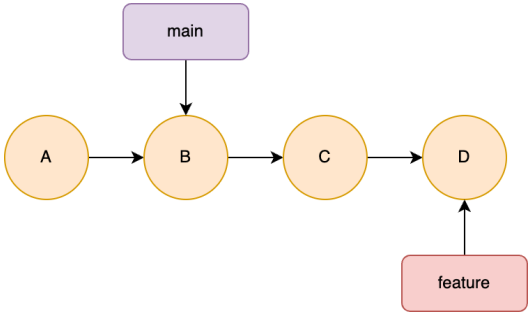
- Executed from the target branch you are merging into:

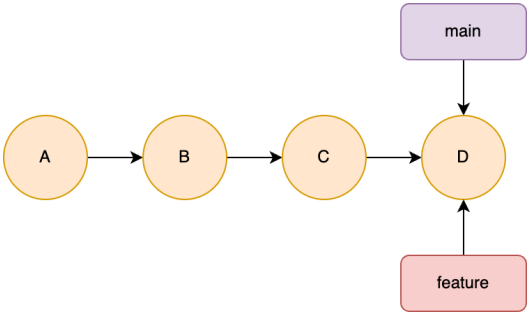
```
git merge <branch-to-merge>
```

- **Merge Methods:**
  - Fast-forward merge
  - 3-way merge

Fast-Forward Merge

- Occurs if the branch being merged into has not diverged from the source branch.



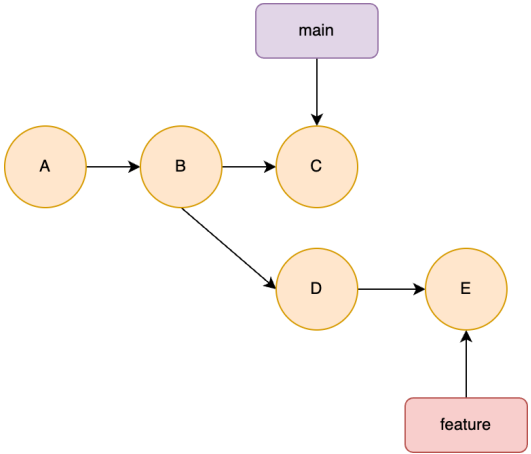


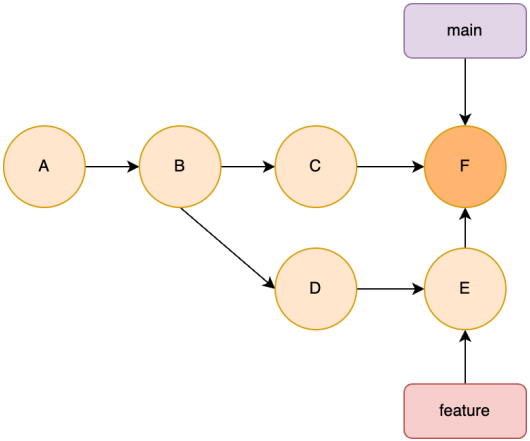
Practice: Fast-Forward Merge

1. Switch to branch `main`. Merge branch `linux-installation` into `main`.

3-Way Merge

- Occurs when branches have diverged (both target and feature branches have independent new commits since splitting).





Practice: 3-Way Merge

1. Switch to branch `main`. Merge branch `refactor` into `main`.